Small Models, Big Tasks: An Exploratory Empirical Study on Small Language Models for Function Calling

Ishan Kavathekar*
International Institute of Information Technology
Hyderabad, India
ishankavathekar31@gmail.com

Raghav Donakanti*
International Institute of Information Technology
Hyderabad, India
raghav.donakanti@students.iiit.ac.in

Ponnurangam Kumaraguru
International Institute of Information Technology
Hyderabad, India
pk.guru@iiit.ac.in

Karthik Vaidhyanathan
International Institute of Information Technology
Hyderabad, India
karthik.vaidhyanathan@iiit.ac.in

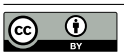
Abstract

Function calling is a complex task with widespread applications in domains such as information retrieval, software engineering and automation. For example, a query to *book the shortest flight from New York to London on January 15* requires identifying the correct parameters to generate accurate function calls. Large Language Models (LLMs) can automate this process but are computationally expensive and impractical in resource-constrained settings. In contrast, Small Language Models (SLMs) can operate efficiently, offering faster response times, and lower computational demands, making them potential candidates for function calling on edge devices. In this exploratory empirical study, we evaluate the efficacy of SLMs in generating function calls across diverse domains using zero-shot, few-shot, and fine-tuning approaches, both with and without prompt injection, while also providing the finetuned models to facilitate future applications. Furthermore, we analyze the model responses across a range of metrics, capturing various aspects of function call generation. Additionally, we perform experiments on an edge device to evaluate their performance in terms of latency and memory usage, providing useful insights into their practical applicability. Our findings show that while SLMs improve from zero-shot to few-shot and perform best with fine-tuning, they struggle significantly with adhering to the given output format. Prompt injection experiments further indicate that the models are generally robust and exhibit only a slight decline in performance. While SLMs demonstrate potential for the function call generation task, our results also highlight areas that need further refinement for real-time functioning.

CCS Concepts

• **Software and its engineering**; • **Computing methodologies** → **Artificial intelligence**; • **General and reference** → **Empirical studies**;

*Both authors contributed equally to this work.



This work is licensed under a Creative Commons Attribution 4.0 International License.
EASE '25, Istanbul, Turkiye
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1385-9/25/06
https://doi.org/10.1145/3756681.3757001

Keywords

Small Language Models, Function Calling, Edge AI, Reliability

ACM Reference Format:
Ishan Kavathekar, Raghav Donakanti, Ponnurangam Kumaraguru, and Karthik Vaidhyanathan. 2025. Small Models, Big Tasks: An Exploratory Empirical Study on Small Language Models for Function Calling. In *Evaluation and Assessment in Software Engineering (EASE '25), June 17–20, 2025, Istanbul, Turkiye.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3756681.3757001

1 Introduction

Advancements in LLMs have enabled strong performance across a wide range of tasks [23, 49]. They are increasingly being adopted in software engineering for tasks such as code generation [31], debugging [39], and API integration [48], as well as in systems for task automation. One critical application for LLMs is function calling and execution. LLMs often rely on extensive cloud-based infrastructure, raising concerns about privacy, latency, and high computational overhead [11, 45]. As a result, SLMs are emerging as a powerful alternative for real-world applications that can efficiently perform specific tasks locally, making them ideal for environments where responsiveness and data security are critical [40]. SLMs can potentially serve as a bridge between user inputs and backend systems in function calling applications, by converting natural language queries into structured function calls [40]. Function calling, though less common than NLP tasks like summarization or translation, is crucial for interpreting and executing user commands. It enables SLMs to perform real-world actions, making it essential in industries requiring timely, accurate responses, such as healthcare, finance, automotive systems, telecom, and smart home automation. However, this intersection of function calling with SLMs is relatively an unexplored and untapped space.

To this end, we conduct an exploratory empirical study on the capability of SLMs to generate function calls. We systematically select 5 SLMs based on a coding capabilities evaluation benchmark and on the number of parameters present in the model and evaluate them on a comprehensive dataset of 60,000 samples spanning diverse functional domains such as technology, entertainment, finance, and more. We evaluate the SLMs using three inference strategies: zero-shot, few-shot, and finetuning and assess their performance across different metrics which capture multiple dimensions of function call generation. Our metrics assess syntactic correctness, semantic

accuracy, and the models' ability to generate structured outputs across various scenarios as well. Additionally, we conduct prompt injection experiments to assess the models' robustness to minor perturbations in the user prompt. Furthermore, we conduct experiments on an edge device to examine the models' practical applicability in constrained computational environments and analyze the relationship between the memory footprint of the model and latency. In summary, our contributions include: (i) an analysis of SLMs with various prompting techniques, including prompt injection, for function calling, (ii) finetuned SLMs with performance analysis, highlighting areas for improvement, and (iii) experiments on edge devices to assess efficiency.

Our findings indicate that certain SLMs demonstrate promising capabilities across various approaches. Additionally, our analysis reveals interesting insights that open new avenues for future research. We provide the finetuned models to support further research and applications. The replication package for our study is here [1].

2 Background

2.1 Small Language Models (SLMs)

SLMs are generative models capable of understanding, processing, and generating text corresponding to the user prompt. These models operate on a much smaller scale compared to LLMs. In contrast to billions of parameters of LLMs, SLMs use significantly less parameters ranging from millions to a few billion. This reduces memory and computation, improving efficiency, accessibility, customizability, and inference speed. These characteristics make SLMs a better alternative for domain-specific, constrained, and low resource setting compared to LLMs.

2.2 Zero-shot, Few-shot, and Finetuning approaches for inference

Zero-shot approach involves leveraging the pre-training of the model and prompting it directly without any other task information. In contrast, few-shot approach involves providing one or more examples of task-answer responses in the prompt itself. The model uses these examples to understand the task and generate the desired output [37]. Since these examples are provided into the model's context, few-shot approach is also known as in-context learning. The number of examples to be provided in the prompt differs according to the task. This choice is also driven by the length of one example and the context size of the model. Zero-shot and few-shot inferencing do not require additional training of the model, making them computationally efficient techniques. Finetuning refers to the retraining of the model on task-specific data to improve it's performance on the task. However, finetuning is computationally more expensive compared to zero-shot and few-shot approaches due to the need for additional training, which involves updating model parameters, and increased memory usage. To address this, techniques like LoRA [20], prompt tuning [24] and prefix tuning [25] are employed. These methods reduce the computational cost significantly, while giving almost the same performance.

[1] https://github.com/Raghav010/Small-Models-Big-Tasks

2.3 GGUF models

GGUF [15] is a format used for storing and running quantized LLMs efficiently on compute constrained devices. Quantization is a compression process that converts the weights of a model from high-precision representation to low-data-precision representation to reduce memory usage and improve computational efficiency [13]. This format supports models with varying sizes and quantization levels, significantly lowering their memory footprint without declining the performance too much. GGUF is a successor to the GGML [14] format with improved quantization methods and metadata management. With larger models being released, GGUF variants of models are a better alternative compared to previous methods such as half-precision models and GGML format.

2.4 Prompt Injection

Prompt injection refers to the manipulation of the user prompt to alter the output of the generative model [28]. This involves appending certain special characters, instructions, or adversarial text to the initial prompt leading to toxic, biased, and undesirable outputs. Prompt injection can be categorized into two types: (i) direct prompt injection and (ii) indirect prompt injection. While direct prompt injection involves directly altering the input of the model [7], indirect prompt injection modifies the external data the model has access to while generation [44]. In this study, we primarily focus on direct prompt injection, since we only focus on function call generation without external data.

3 Related Work

LLMs are increasingly being employed for various Software Engineering (SE) tasks such as automated code generation and repair [22], requirement analysis and design [9], intelligent project management [4] and more, reflecting their widespread adoption across various applications. One such task is function calling. Over the years, researchers have explored various strategies to enable LLMs to generate accurate function calls. These methods can be broadly categorized into two main strategies. The first one involves prompting the model efficiently to leverage their knowledge acquired during training. This can be achieved by in-context learning [42] or using techniques like ReAct [47] which combine reasoning and action to guide the models generate better responses. The second strategy involves finetuning the models over a diverse set of datasets or a domain specific dataset to enhance their performance. Models like Gorilla [34], Nous-Hermes-13b [32] and, ToolAlpaca [38] are trained on synthetic data generated by GPT-4 and GPT-3.5. On the other hand Granite-20B [1] has been finetuned on API-Blend [6] which consists of five diverse API datasets. Several benchmarks such as ToolBench [18], ComplexFuncBench [50], Berkley Function calling leaderboard (BFCL) [46] have been introduced to evaluate function calling capabilities of LLMs. These benchmarks assess various factors like real API Response, multi-step function calls, and more. However, previous work mainly focuses on finetuning large models and benchmarks which are not specifically designed to evaluate the challenges associated with smaller models. Our work addresses this gap by conducting a fine-grained empirical study on SLMs, analyzing their performance, and also providing smaller finetuned models.

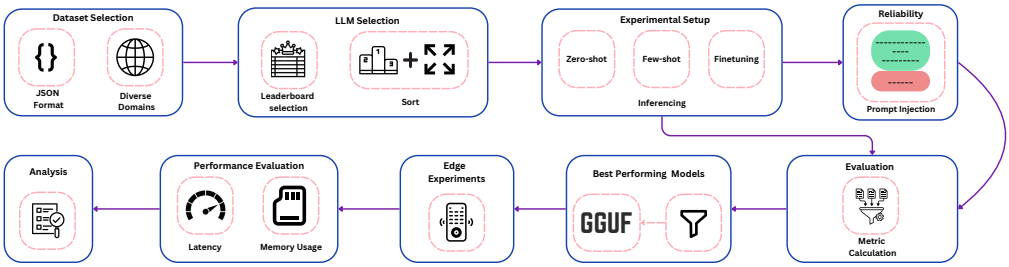


Figure 1: Overview of the study design illustrating different components of the research methodology.

4 Study Design

4.1 Goal

This paper aims to understand the efficacy of SLMs in generating function calls when presented with a set of function descriptions and an user query as illustrated in Figure 1. We employ the Goal-Question-Metric approach [10] to formally define the objective as follows:

Analyze the effectiveness of Small Language Models
For the purpose of generating function calls
With respect to accuracy, reliability, and robustness
From the viewpoint of researchers and developers
In the context of function invocation tasks.

4.2 Research Questions

In this section, we explore key research questions aimed at evaluating the robustness and practicality of SLMs for function call generation. We aim to answer the following research questions:

RQ_1: Can SLMs be successfully employed to generate function calls given a task and scenario in a zero-shot setting?

Through this question, we aim to evaluate whether SLMs are able to generate accurate and precise function calls given the function descriptions and user query without being exposed to the task. This gives us a chance to evaluate the inherent abilities of the SLMs, solely based on their pre-trained knowledge.

RQ_2: How does few-shot approach affect the SLM's ability to generate function calls?

Providing examples of similar questions in the prompt enhances the performance of generative models. These examples help the SLM understand the nuances of the task and guides it in generating task-appropriate responses. We examine how providing examples of task-response pairs affects the performance of the SLMs.

RQ_3: Does fine-tuning SLMs enhance its capability of generating function calls ?

Finetuning models on the task-specific data has shown to significantly improve their capabilities. Hence, we examine how does it impact the performance of SLMs in the task of function call generation.

By examining the performance of SLMs in the first three research questions, we also aim to identify how prompt injection influences the robustness of zero-shot (RQ_1), few-shot (RQ_2), and fine-tuning (RQ_3) outcomes. A successful demonstration of prompt injection robustness would indicate that SLMs can maintain their functional integrity and adaptability in dynamic environments, ultimately supporting their reliable [2] integration into automated systems.

RQ_4: How do SLMs perform in generating function calls when deployed on an edge device?

Edge devices operate on constrained resources and process the input locally, leading to a lower latency and memory usage of the model. We aim on assess how deploying these SLMs to an edge device affects their performance.

4.3 Experiment Subject

Models considered in the study: We evaluated five SLMs for their ability to generate function calls across various settings. These models were selected from the EvalPlus benchmark [27], a code synthesis evaluation framework. This benchmark may reflects a model's ability to compose functions accurately since the paradigms of code synthesis and function call generation are closely aligned. The top five models upto 4B parameters were chosen for the study. This choice was made with consideration for the experiments conducted on edge device. The EvalPlus benchmark is an evolving leaderboard and the models evaluated in the study ranked at the top five positions according to our requirements, at the time of conducting the study. Table 1 summarizes the details of the selected models as available on HuggingFace.

Model	Size	Context Length
Deepseek-coder-1.3B-instruct [17]	1.35B	128K
Phi-3-mini-4k instruct [2]	3.82B	4K
Phi-2 [21]	2.78B	2048
Starcoder2-3B [30]	3.03B	16K
Stable-code-3B [35]	2.8B	16K

Table 1: SLMs used in the study, along with their parameter size and context length.

Dataset: The Salesforce XLAM Function Calling dataset [29], available on Hugging Face, is a high-quality resource specifically designed for function calling tasks. With 60,000 samples, it provides a substantial and diverse dataset covering a wide range of domains and scenarios. Derived from the ToolBench [36] dataset, it benefits from careful curation and reflects real-world function calling contexts, enhancing the practical applicability of our research findings. Each datapoint of the dataset contains three components: *(i)*

[2]In this study, reliable refers to the reliability of the responses generated by SLMs, rather than the reliability of the system itself.

query: the problem statement, *(ii) tools*: name, description and parameters of the available functions ,and *(iii) answer*: the correct function call. Figure 2 illustrates an example from the dataset. This dataset's adherence to JSON format provides key advantages for our study. The structured JSON format facilitates automated processing and evaluation of model outputs, streamlining our research workflow. Furthermore, JSON's language-independent nature makes it convertible to various programming languages, enhancing the versatility of our function calling solutions. To ensure compatibility with the context length constraints of our smaller transformer models (1.35B–3.82B parameters), we curated a subset of the Salesforce XLAM Function Calling dataset within each model's limits (Table 1) for a fair performance comparison. We then split this dataset into a test set (5,000 samples) and a finetuning set (55,000 samples).



```
{
  "query": "Find the product of the first five prime numbers.",
  "tools": [
    {
      "name": "math_toolkit.sum_of_multiples",
      "description": "Find the sum of all multiples of specified
                      numbers within a specified range.",
      "parameters": {
        "lower_limit": {
          "type": "int",
          "description": "The start of the range (inclusive).",
          "required": true
        },
        "upper_limit": {
          "type": "int",
          "description": "The end of the range (inclusive).",
          "required": true
        },
        "multiples": {
          "type": "list",
          "description": "The numbers to find multiples of.",
          "required": true
        }
      }
    },
    {
      "name": "math_toolkit.product_of_primes",
      "description": "Find the product of the first n prime numbers.",
      "parameters": {
        "count": {
          "type": "int",
          "description": "The number of prime numbers to multiply together.",
          "required": true
        }
      }
    }
  ],
  "answers": [
    {
      "name": "math_toolkit.product_of_primes",
      "arguments": {
        "count": 5
      }
    }
  ]
}
```


Figure 2: Sample datapoint from the Salesforce-XLAM function-calling dataset, illustrating its components: user query, available tools, and the answer.

4.4 Experimental Procedure

In this subsection, we provide a comprehensive overview of the experimental setup and methodology. All inferences, except edge deployments, were conducted on a single-GPU setup using either a GTX 1080 Ti or a RTX 2080 Ti at float 16 precision on the test set.

Zero-shot approach: For zero-shot prompting experiments, we employ a standardized prompt template to organize the query, tools, and expected answers, ensuring consistency in input formatting across all samples. Additionally, the prompt also includes explicit instructions for the model to produce output in a specified JSON format, facilitating structured and machine-readable responses. The experiments were conducted with a temperature setting of 0 to minimize randomness and ensure deterministic outputs.

Prompt for Zero-shot approach

Task Instruction: System prompt and description of the function call generation task
Tools Description: Description of all tools available to complete the user query
Format Instruction: Structure and format for the generated output
User Query

Few-shot approach: For the few-shot prompting experiments, we used the same standardized prompt template as in the zero-shot setup, including explicit instructions for the model to produce outputs in the specified JSON format. In addition to the query, tools, and expected answers, the prompt was appended with three task-specific examples (3-shot prompting) to guide the model in understanding the desired behavior. Providing a small number of examples (2-5) can significantly improve model performance [8]. Considering the size of the dataset and the context length limitations of the model, we provide three examples by assessing the context length of the models and example length. These examples were carefully selected to cover representative scenarios within the test set and remain constant for all datapoints.

Prompt for Few-shot approach

Task Instruction: System prompt and description of the function call generation task
Tools Description: Description of all tools available to complete the user query
Format Instruction: Structure and format for the generated output
Sample Examples: Three query-response examples of the function call generation task
<SAMPLE EXAMPLE 1> <SAMPLE EXAMPLE 2> <SAMPLE EXAMPLE 3>
User Query

Finetuning: For the fine-tuning experiments, we utilize a single-GPU RTX A5000 setup, and employ LoRA (Low-Rank Adaptation) to adapt model parameters effectively while keeping minimal additional overhead. The LoRA configuration included a rank of 8, which specifies the dimensionality of the low-rank matrices used for parameter updates, enabling efficient learning without modifying the entire model. The scaling factor (alpha) was set to 8, amplifying the updates in the low-rank space to balance learning stability and effectiveness. A dropout rate of 0.05 was applied to the LoRA weights to introduce regularization and mitigate overfitting. LoRA was applied to all linear layers in the model, including the query and value projection layers. The fine-tuning process was conducted on the 55,000-sample subset over 2 epochs. Training used a batch size of 2 per device, with evaluation at a batch size of 4, and gradient accumulation steps set to 4 to effectively increase the overall batch size. The learning rate was 2e-5. Training was performed at bf16 precision for faster computation and reduced memory usage. All fine-tuned models, along with the code, are provided in supplementary material for reproducibility.

Prompt Injection: We evaluate the robustness of SLMs under zero-shot, few-shot and finetuning settings by performing prompt injection. This involves appending a string of non-sensical random characters (including alphanumeric, special and unicode characters)into the user prompt. This assesses the model's ability to adhere

to the original task and ignore any noise in the input.

Prompt for Prompt Injection experiment

Task Instruction: System prompt and description of function call generation task
Tools Description: Description of all the tools available to complete the user query
Format Instruction: Structure and format for the generated output
(Sample examples are included only in the case of few-shot experiments)
Sample Examples: Three query-response examples of the function call generation task
<SAMPLE EXAMPLE 1> <SAMPLE EXAMPLE 2> <SAMPLE EXAMPLE 3>
User Query + $l3_aq - a$" <: $11|E > 3vn8IsdeF\$rnjQ\&$

Edge Device experiments: Due to it's ability to work within the memory and compute constraints of the edge device, we employ GGUF variants of the selected models for the edge experiments. We utilize the 4-bit quantization Q4_K_M format for quantizing the model as it ideally balance between compactness and performance. For edge experiments, we choose, *Qualcomm QIDK (Qualcomm Innovation Development Kit)*, equipped with Snapdragon ® 8 Gen 2 processor, Adreno GPU. It has the system-on-chip (SoC) that powers various commercial smart phones. We perform two sets of experiments on the edge device. First, we select the best performing model-setting pairs from the aforementioned experiments (zero-shot, few-shot and finetuning). We convert these models to the GGUF format to deploy it on the edge device. We exclude the prompt injection experiments in the edge setting. Due to the compute constraints of the edge device, we use a subset of 100 datapoints from the test set. These datapoints were selected randomly from the dataset ensuring unbiased selection. We then calculate the average latency and memory usage of these GGUF models and comparing the values with half precision models for the function calling task.

4.5 Metrics

Despite advances in code evaluation. there are no metrics that comprehensively evaluate function call generation. While metrics such as ROUGE [26], BLEU score [33] and METEOR [5] exist for assessing structural correctness and code similarity, they do not serve the purpose of our study as we focus on function call generation. Abstract Syntax Trees (AST) are specifically used to analyze the structural relationship between different components of the code. However, since our study focuses exclusively on function call generation rather than full function generation, we do not include AST analysis. Hence, to evaluate the performance of models on the function calling task, we introduce five novel metrics to assess both the syntactic correctness of the model outputs and the semantic accuracy of the function calls, arguments, and values.

JSON Parsability measures whether the model's output adheres to a valid and expected JSON structure. JSON parsability ensures the syntactic correctness of the output, which is essential for downstream tasks that rely on structured data.

Let O_i represent the output for the i-th data point, and $\mathcal{J}(O_i)$ be an indicator function that is 1 if O_i is a valid JSON object, and 0 otherwise. The overall JSON parsability metric is calculated as:

$$\text{JSON Parsability} = \frac{1}{N}\sum_{i=1}^{N}\mathcal{J}(O_i) \tag{1}$$

Where N is the total number of data points. All the following metrics are computed only on the data points where $\mathcal{J}(O_i)$ is 1.

Task Accuracy evaluates the overall correctness of the function calls given a task/query. Task Accuracy encompasses a broad range of function facets, capturing the correctness of both function selection and formulation, reflecting the performance in areas that the other metrics individually focus on. Essentially, it assesses the model's ability to understand the task correctly and to generate the appropriate function calls completely. For a given data point, let: T_{true} be the set of true function calls (ground truth), T_{pred} be the set of predicted function calls. Task Accuracy is defined as the F1-score between T_{pred} and T_{true}. The overall Task Accuracy is the average across all data points.

Correct Ratio serves as a stricter evaluation of the model's performance by measuring the proportion of data points where the Task Accuracy is exactly 1, making it an indicator of the model's ability to deliver fully correct outputs. Unlike broader metrics, Correct Ratio only considers cases where the predicted function calls perfectly match the ground truth in both function selection and formulation, with no errors or omissions.

Function Selection Performance (FSP) measures the model's ability to select the correct function(s) from a given set of available functions. By focusing specifically on function selection, FSP isolates the model's capacity to correctly interpret the intent of the task and map it to the right tool or function. Let F_{true} be the set of function names in the ground truth, and F_{pred} be the set of function names predicted by the model. FSP for a given data point is calculated as:

$$\text{FSP} = \frac{|F_{\text{true}} \cap F_{\text{pred}}|}{|F_{\text{true}}|} \tag{2}$$

Where $|F_{\text{true}} \cap F_{\text{pred}}|$ represents the number of correctly predicted function names. The overall FSP is the average across all data points.

Argument Completeness Score (ACS) assesses the model's ability to include the necessary argument names in its function calls. A high ACS indicates that the model understands the full scope of the function's requirements. Let: A_{true} be the set of argument names in the ground truth, A_{pred} be the set of predicted argument names. ACS for a function is the F1 score between A_{true} and A_{pred}. The overall ACS is calculated as the average over all functions belonging to $|F_{\text{true}} \cap F_{\text{pred}}|$ over all data points.

Argument Value Correctness (AVC) further refines the evaluation by measuring the correctness of the values assigned to the arguments. While ACS evaluates whether the correct argument names are provided, AVC assesses whether the model assigns the correct values to those arguments. AVC is a more stringent measure of the model's understanding of the details of the task.

For arguments that have been correctly predicted (as identified by ACS), let $v_{\text{true}}(a)$ and $v_{\text{pred}}(a)$ be the ground truth and predicted values for argument a. The AVC for a function is given by:

$$\text{AVC} = \frac{1}{|A_{\text{correct}}|}\sum_{a \in A_{\text{correct}}} \mathbb{I}(v_{\text{true}}(a), v_{\text{pred}}(a)) \tag{3}$$

Where A_{correct} is the set of argument names predicted correctly, and $\mathbb{I}(\cdot)$ is the indicator function that equals 1 if the values match and 0 otherwise. The overall AVC is calculated as the average over all functions belonging to $|F_{\text{true}} \cap F_{\text{pred}}|$ over all data points.

5 Results

RQ$_1$: Can SLMs be successfully employed to generate function calls given a task and scenario in a zero-shot setting?
While testing the models in zero-shot setting, we observe that most of the models fail to generate responses adhering to the JSON format even after explicitly mentioning it in the input prompt. As shown in Table 2, only Deepseek-Coder-1.3b-instruct attains a non-zero value in JSON parsability, allowing us to further analyze the responses. Deepseek-Coder generates JSON parsable responses with a success rate of only 7.34%. We observe that it struggles to accurately select the correct function, arguments, and argument values from the provided function descriptions, resulting in a task accuracy of only 1.11%. Manual inspection of 200 responses from models producing zero-values in Table 2 reveals a pattern that these SLMs struggle with uncontrolled generation. While some examples generate correct answer, they often initiate a new unrelated query. Other instances fail to generate proper JSON structures, resulting in incorrect function calls. We discuss this further in Section 6.

The results indicate that prompt injection impacts the models' responses. Although some cases exhibit a decline in JSON parsability and ACS metrics, it is noteworthy that metrics such as FSP, AVC, and overall task accuracy show an improvement.

Main findings of RQ1: SLMs fail to generate accurate function calls in a zero-shot setting. Most models fail to generate responses in compliance with the given format, making the responses unparsable and incorrect. Deepseek-Coder is the only model which generates JSON parsable responses, but such responses remain very limited. Prompt injection experiments demonstrate a slight decline in performance, but the overall impact remains minimal given the poor baseline performance.

RQ$_2$: How does few-shot approach affect the SLM's ability to generate function calls?
Table 2 highlights the impact of providing a few task-specific examples on function call generation. Deepseek-Coder demonstrates a substantial improvement, with a 67–80% increase in metrics such as JSON parsability, FSP, ACS, and AVC, leading to a task accuracy of 55.65%. However, Phi-3-Mini, Phi-2, and StarCoder continue to struggle in generating JSON-parsable responses. Stablecode performs poorly with a task accuracy of only 0.11%.

Main findings of RQ2: A significant increase in performance can be observed in the few-shot setting. However, many models are unable to perform better even after providing examples. Prompt injection results in notable decline in performance.

Prompt injection significantly decreases the performance of the Deepseek-Coder model by 13-16%. Small perturbations made in the input disrupt the model's ability to select the function, arguments and argument values. Consequently, there is 13% decline in the task accuracy, highlighting the model's vulnerability towards small modifications made in the prompt.

RQ$_3$: Does fine-tuning SLMs enhance it's capability of generating function calls ?
Fine-tuning produces mixed performance improvements across models. While Deepseek-Coder and Phi-3-mini show a significant increase in performance, other models fail to achieve non-zero values. As shown in Table 2, Deepseek-Coder achieves a substantial increase in JSON Parsibility, improving from 7.34% in the zero-shot setting to 89.38% in the few-shot setting and reaching 99.44% after fine-tuning. Similarly, Phi-3-mini shows a notable jump, with JSON parsability increasing from 0% in zero-shot and few-shot settings to 99.62% after fine-tuning. Additionally, Phi-3-mini is able to select the correct parameters of the function call reflected by high FSP, ACS and AVC metrics. However, the performance of other models, such as Phi-2, Starcoder, and Stable-code, remains zero even after fine-tuning. This can be attributed to persistent errors in their output structures, making their JSON responses unparsable. It should be noted that Phi-3-mini beats Deepseek-Coder in task accuracy even though Deepseek-Coder performed better in zero-shot and few-shot setting. This result highlights the impact of finetuning models on task-specific data. Furthermore, we observe that finetuning improves robustness against prompt injection attacks. For instance, we observe a slight decline of just 1-2% in performance across metrics compared to a decrease of 10-15% in few-shot setting.

Main findings of RQ3: Finetuned models perform significantly better than zero-shot and few-shot setting. The models are able to accurately select the function parameters leading to a higher task accuracy and correct ratio. Furthermore, prompt injection has little effect on model performance.

RQ$_4$: How do SLMs perform in generating function calls when deployed on an edge device?
For edge device experiments, we evaluated the models using a subset of 100 prompts due to resource constraints. As shown in Table 3, Deepseek-Coder achieves the best performance in the few-shot setting with a task accuracy of 44.97%, outperforming both its zero-shot (32.5%) and fine-tuned (35.7%) configurations. This trend is evident across all metrics including correct ratio. On edge devices, models demonstrate considerable latency as shown in Table 5. Deepseek-Coder exhibits the lowest latency across all settings, ranging from 61.10s to 70.55s. In contrast, Phi-3-mini shows significantly higher latency, requiring 364.51s in zero-shot and 335.58s in few-shot settings. Other models demonstrate intermediate latency values, ranging from 140.29s to 267.65s across different settings. Notably, latency is considerably higher on edge devices compared to server deployments. For instance, Deepseek-Coder, which takes 5.59s in zero-shot and 3.22s in few-shot on a server, experiences an over 10x increase in latency when deployed on an edge device. Moreover, the latency trends between settings differ. While fine-tuned models exhibit higher latencies on servers due to increased processing complexity, few-shot settings incur greater latencies on edge devices. Beyond latency, memory constraints further impact edge deployments. As shown in Table 4, models consume significantly more memory on servers, with Deepseek-Coder requiring 5385.89MB compared to 1678.18MB on an edge device. Similar trends hold for most other models, achieving around a 5x decrease in memory usage when deployed on the edge.

Main findings of RQ4: Edge device deployment shows that Deepseek-Coder achieves the best balance between performance, latency and memory usage. While few-shot learning yields better accuracy, it comes with slightly increased latency compared to other settings. We also observe that models consume about 5x less memory on edge devices than on servers due to GGUF.

Metric	Model	Zero-shot		Few-shot		Finetuned	
		Without Prompt-injection	With Prompt-injection	Without Prompt-injection	With Prompt-injection	Without Prompt-injection	With Prompt-injection
JSON Parsibility	Deepseek-coder-1.3B-instruct	**0.0734**	**0.0140**	**0.8938**	**0.7268**	0.9944	0.9906
	Phi-3-mini-4k-instruct	0.0000	0.0000	0.0000	0.0084	0.9962	**0.9939**
	Phi-2	0.0000	0.0000	0.0000	0.0002	0.0000	0.0000
	Starcoder2-3B	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Stable-code-3B	0.0000	0.0000	0.0058	0.0060	0.0000	0.0000
Task Accuracy	Deepseek-coder-1.3B-instruct	**0.0111**	**0.0527**	**0.5565**	**0.4289**	0.8543	0.8404
	Phi-3-mini-4k-instruct	0.0000	0.0000	0.0000	0.0012	**0.8727**	**0.8598**
	Phi-2	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Starcoder2-3B	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Stable-code-3B	0.0000	0.0000	0.0011	0.0009	0.0000	0.0000
Correct Ratio	Deepseek-coder-1.3B-instruct	**0.0470**	**0.0098**	**0.468**	**0.3384**	0.8074	0.7866
	Phi-3-mini-4k-instruct	0.0000	0.0000	0.0010	0.0008	**0.8328**	**0.8210**
	Phi-2	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Starcoder2-3B	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Stable-code-3B	0.0000	0.0000	0.001	0.0006	0.0000	0.0000
FSP	Deepseek-coder-1.3B-instruct	**0.0139**	**0.0723**	**0.8846**	**0.7209**	0.9918	0.9859
	Phi-3-mini-4k-instruct	0.0000	0.0000	0.0000	0.0031	**0.9936**	**0.9901**
	Phi-2	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Starcoder2-3B	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Stable-code-3B	0.0000	0.0000	0.0017	0.0016	0.0000	0.0000
ACS	Deepseek-coder-1.3B-instruct	**0.0699**	**0.0136**	**0.8404**	**0.6783**	0.9664	0.9596
	Phi-3-mini-4k-instruct	0.0000	0.0000	0.0000	0.0031	**0.9700**	**0.9652**
	Phi-2	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Starcoder2-3B	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Stable-code-3B	0.0000	0.0000	0.0017	0.0016	0.0000	0.0000
AVC	Deepseek-coder-1.3B-instruct	**0.0129**	**0.0614**	**0.6811**	**0.5488**	0.9065	0.8973
	Phi-3-mini-4k-instruct	0.0000	0.0000	0.0000	0.0015	**0.9172**	**0.9087**
	Phi-2	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Starcoder2-3B	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
	Stable-code-3B	0.0000	0.0000	0.0012	0.0010	0.0000	0.0000

Table 2: Model performance across different metrics and configurations, with and without prompt injection.

Model and Setting	JSON Parsibility	Correct Ratio	FSP	ACS	AVC	Task Accuracy
Deepseek-coder-1.3B-instruct zero-shot	0.4000	0.3000	0.4000	0.3871	0.3633	0.3250
Deepseek-coder-1.3B-instruct few-shot	0.6000	0.4000	0.6000	0.5859	0.5312	0.4497
Deepseek-coder-1.3B-instruct finetuned	0.4100	0.3200	0.4067	0.4012	0.3743	0.3570
Phi-3-mini-4k-instruct finetuned	0.3900	0.3200	0.3800	0.3689	0.3517	0.3417

Table 3: Model performance for the best performing model-setting pair on edge device.

6 Discussion

Lessons Learned: Our study reveals several key insights into the function calling capabilities of SLMs. First, zero-shot and few-shot performance across all evaluated models is generally poor, with Deepseek-Coder being the only model that can generate function calls in a structured format. Finetuning the models significantly improves the model performance. However, we still observe many models with zero values in Table 2. Figure 3 illustrates examples of Deepseek-Coder and Phi-3-mini responses across different settings. Common problems found across different settings include uncontrolled generation and an inability of models to produce responses in the specified format. Figure 3(b), 3(e) and 3(f) illustrate instances where a model generates a correct answer but fails to adhere to the required JSON format or continues generating text beyond the answer. However, these responses do not show any pattern, making it difficult to come up with a systematic method to process these responses for further use. Hence, we do not investigate this further. Additionally, the vulnerability of models in the few-shot setting to prompt injection remains a concern, as minor perturbations lead to performance degradation. On the other hand fine-tuned models exhibit greater robustness against prompt perturbations, indicating improvements in structured response adherence. This necessitates that more work needs to be done around building small models which can handle these perturbations and also exploring the use of input-validators which can clean the prompt for inaccuracies before passing it on to the model. It would require a mix of SE and NLP expertise to make this possible. Deploying SLMs on edge devices requires balancing latency, memory, and performance. As shown in Figure 4, edge latencies often exceed server latencies by an order of magnitude, necessitating model optimizations and efficient inference techniques like flash attention [12] or hardware acceleration. Sustained usage also faces power and temperature constraints, which can be mitigated through NPUs and specialized accelerators for improved efficiency and thermal management.

Model	Server (fp16)	Edge (GGUF)
Deepseek-coder-1.3B-instruct	2,570.24	1,678.18
Phi-3-mini-4k-instruct	7,642.16	3,990.97
Phi-2	5,304.32	2,443.01
Starcoder2-3B	5,775.36	2,051.05
Stable-code-3B	5,335.04	3,058.95

Table 4: Memory usage (MB) for models across settings.

Future Research in NLP: Our findings highlight several avenues for further research in improving SLMs for generation tasks. The issue of uncontrolled generation and improper formatting necessitates advancements in decoding strategies. Research into constrained decoding techniques [41], such as grammar-constrained sampling and structured output models, could help mitigate these challenges. Another point to note is that we experiment with a simple prompt injection attack and observe a significant decline in model performance. A more complex attack such as multi-turn

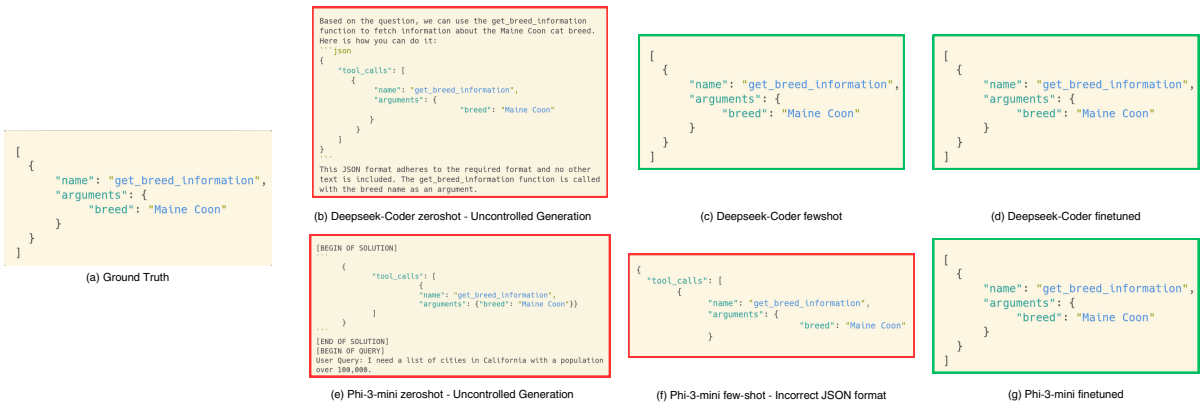


Figure 3: Responses of Deepseek-Coder and Phi-3-mini across settings highlighting incorrect (red) and correct (green) outputs.

prompt injection [3] has the potential to manipulate the model into leaking sensitive system information. Hence, it is essential to develop robust defense mechanisms to mitigate the impact of such attacks. Building task-specific fine-tuned models may present challenges in generalizability across different function calling domains. Future research could explore cross-task applicability, and leveraging synthetic data generation for domain specific datasets.

Future Research for SE: SLMs present promising opportunities for efficient and scalable solutions in software engineering. Fine-tuned models have demonstrated strong performance in function calling tasks, highlighting their potential for real-world applications. Additionally, our study shows prompt injection has minimal impact on their effectiveness. Researchers should develop robust, fine-tuned SLMs capable of handling diverse scenarios, improving reliability and adaptability in function call generation. Studying generalizability across SE lifecycle tasks and resilience to attacks can further enhance their real-world viability. This will help bridge the gap between experimentation and practical applications. SLMs also present an opportunity is in terms of re-imagining the integration of models with software systems to make them more efficient. Most LLM-powered applications rely on a single LLM to handle and solve multitude of tasks. In contrast, SLM research has focused on building task-specific models that execute specialized functions with greater efficiency. This can be leveraged to engineer sustainable software systems by decomposing large complex tasks and routing them to various specialized SLMs, maintaining overall performance while significantly reducing computational costs. This shift towards task-specific SLMs not only enhances efficiency but also opens avenues for hybrid deployment strategies[19]. Techniques such as edge-cloud collaboration, where initial inference occurs on-device and complex queries are offloaded to the cloud, can reduce latency while maintaining robustness. Techniques such as model compression [51], quantization [13], and knowledge distillation [16] can help optimize computational efficiency, enable faster inference. However, their effectiveness for software engineering tasks is underexplored, presenting a direction for future research.

Setting	Model	Zero-shot	Few-shot	Finetuned
Server (fp16)	Deepseek-coder-1.3B-instruct	5.59	3.22	3.91
	Phi-3-mini-4k-instruct	35.23	39.14	32.88
	Phi-2	22.19	28.97	62.40
	Starcoder2-3B	32.05	33.25	60.32
	Stable-code-3B	22.95	27.67	31.91
Edge (GGUF)	Deepseek-coder-1.3B-instruct	64.87	70.55	61.10
	Phi-3-mini-4k-instruct	364.51	335.58	124.76
	Phi-2	153.67	250.77	140.29
	Starcoder2-3B	246.63	262.92	267.65
	Stable-code-3B	214.89	265.90	175.77

Table 5: Latency of models (sec) for models across settings.

Implications for Practice: Our findings indicate that deploying SLMs for function calling in real-world applications requires careful consideration of performance, reliability, and integration challenges. Real-world environments impose higher expectations on performance and reliability, as application developers require models that can be safely integrated into production systems. Nonetheless, SLMs offer advantages in real-time applications where low latency is crucial, reducing inference costs and supporting Greening AI initiatives by lowering energy use and carbon footprint. However, one challenge remains-inconsistent adherence to structured formats, which presents challenges for integration into production systems. Most models do not adhere to the specified format, necessitating engineering solutions for enhancing adaptability. Approaches such as wrappers that enforce structured outputs, response format checkers, and guardrails can help mitigate malformed outputs. Prompt engineering, which involves carefully designing instructions to align with model-specific prompting formats, can improve structured output generation. However, perturbations to the prompt via prompt injection attacks can render the outputs unusable. In contrast, tools such as Typechat and Guidance can be used to steer the generative models to generate structured outputs. TypeChat enables TypeScript-based interactions with models, using re-prompting and the TypeScript compiler for output validation. In contrast, Guidance combines prompt engineering, constrained token generation, and re-prompting to improve control and precision in model responses. We believe that developers can greatly benefit from these tools for generating better responses and refining the malformed ones. A key deployment consideration is choosing between few-shot prompting and fine-tuning. Few-shot prompting offers flexibility, allowing easy adaptation to backend updates through prompt modifications. Fine-tuning enhances task performance and robustness to prompt

perturbations but demands higher compute. The choice depends on balancing adaptability with performance needs. Despite their potential in function-calling systems, widespread adoption of SLMs is limited by challenges such as inefficient deployment frameworks and the heterogeneous nature of edge devices, complicating real-world integration. Additionally, ensuring reliable performance and low-latency execution in dynamic, resource-constrained environments is a persistent hurdle. Tackling these challenges calls for a stronger collaboration between the SE and NLP communities.

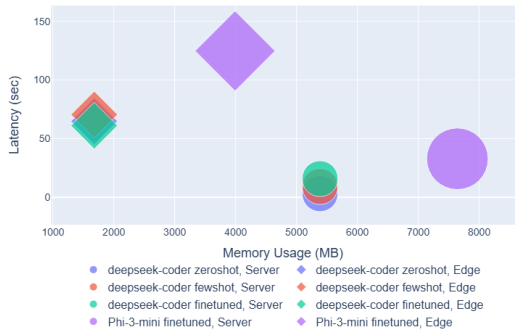


Figure 4: Comparison of latency (in sec) and Memory usage (in MB) of Models on edge and server environments.

7 Threats to validity

In this section we discuss the threats to validity of our study. We follow the categorization provided by Wohlin et al. [43] and explain how we try to mitigate them.

Internal Validity: The selection of models from the EvalPlus leaderboard was based on their demonstrated coding abilities, reducing the risk of bias in model selection. This ensures that our analysis is focused on state-of-the-art models. A threat to validity can arise from the metrics used in the study. To the best of our knowledge, no standard metrics exist to extensively evaluate function calls. Existing metrics like ROUGE, BLEU, and METEOR focus on the textual syntax and semantics, making them unfit for our usecase. Hence, we define metrics that evaluate various aspects of a function definition.

External Validity: The rapid release of newer SLMs with advanced capabilities make it unfeasible to consider all the models for the study. Hence, we carefully select the top performing models from the EvalPlus leaderboard which match our requirements. Another threat to validity is the possibility that the models may have been exposed to the test data during training. However, since most models do not publicly disclose detailed information about their training data, we are unable to mitigate this issue. We exclusively consider the QIDK device for our edge experiments. However, performance results may be different between other edge devices with different hardware configurations. Future work could explore how these models perform on a broader range of edge devices.

Construct Validity: A potential threat to construct validity can arise from the dataset used to evaluate the models. To mitigate this, we select the salesforce-xlam function calling dataset which contains a diverse set of real-world function calls from domains such as finance, technology, health care, sports and more. We choose 100 datapoints from the dataset for our edge device experiments. A threat to validity might stem from this if the subset is not a representative of the whole dataset. To mitigate this, we randomly select the datapoints to ensure a balanced sample. Additionally, the study's metrics comprehensively evaluate the models' function call generation capabilities across multiple dimensions.

8 Conclusion

In this study, we explore the ability of Small Language Models to generate function calls. We evaluate five top ranked SLMs from the EvalPlus leaderboard across various experimental settings such as zero-shot, few-shot, finetuning, and on an edge device. In addition, we assess the robustness of these models by performing prompt injection experiments. We find that SLMs struggle to generate function calls autonomously, their performance improves with post-processing, albeit with some degradation due to prompt injection. Furthermore, we observe that high performance of a model on a server does not necessarily translate into edge device settings. Furthermore, we provide the finetuned models from our study and outline a path forward for the NLP and SE communities to enhance function call generation in practical scenarios.

Future work can involve extending this study to language specific function call generation, providing insights on model performance across programming languages. Exploring additional edge devices and assessing various adversarial attacks would provide a more comprehensive robustness evaluation. Another potential direction is leveraging LLM-based agents and multi-agent systems to generate function calls and execute them.

Acknowledgments

We sincerely thank Qualcomm Inc. and AlphaGrep for their generous funding for this study through supporting IIIT-Hyderabad, India.We thank Sujay Belsare and Ananth Yegavakota for assistance with edge device experiments, and Kunal Bhosikar, Hiya Bhatt, Shrikara A, and members of the SERC and Precog groups at IIIT-Hyderabad for their feedback. ChatGPT was utilized to refine some sections of this work including text, tables, code, etc.

References

[1] Ibrahim Abdelaziz, Kinjal Basu, Mayank Agarwal, Sadhana Kumaravel, Matthew Stallone, Rameswar Panda, Yara Rizk, GP Bhargav, Maxwell Crouse, Chulaka Gunasekara, et al. 2024. Granite-function calling model: Introducing function calling abilities via multi-task learning of granular tasks. *arXiv preprint arXiv:2407.00121* (2024).

[2] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, and Arash Bakhtiari et al. 2024. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv:2404.14219 [cs.CL] https://arxiv.org/abs/2404.14219

[3] Divyansh Agarwal, Alexander Fabbri, Ben Risher, Philippe Laban, Shafiq Joty, and Chien-Sheng Wu. 2024. Prompt Leakage effect and mitigation strategies for multi-turn LLM Applications. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, Franck Dernoncourt, Daniel Preoţiuc-Pietro, and Anastasia Shimorina (Eds.). Association for Computational Linguistics, Miami, Florida, US, 1255–1275. doi:10.18653/v1/2024.emnlp-industry.94

[4] Muhammad Azeem Akbar, Arif Ali Khan, Najmul Islam, and Sajjad Mahmood. 2024. DevOps project management success factors: A decision-making framework. *Software: Practice and Experience* 54, 2 (2024), 257–280.

[5] Satanjeev Banerjee and Alon Lavie. 2004. Meteor: an automatic metric for MT evaluation with high levels of correlation with human judgments. *Proceedings of ACL-WMT* (2004), 65–72.

[6] Kinjal Basu, Ibrahim Abdelaziz, Subhajit Chaudhury, Soham Dan, Maxwell Crouse, Asim Munawar, Sadhana Kumaravel, Vinod Muthusamy, Pavan Kapanipathi, and Luis A Lastras. 2024. API-BLEND: A Comprehensive Corpora for Training and Benchmarking API LLMs. *arXiv preprint arXiv:2402.15491* (2024).

[7] Hezekiah J Branch, Jonathan Rodriguez Cefalu, Jeremy McHugh, Leyla Hujer, Aditya Bahl, Daniel del Castillo Iglesias, Ron Heichman, and Ramesh Darwishi. 2022. Evaluating the susceptibility of pre-trained language models via handcrafted adversarial examples. *arXiv preprint arXiv:2209.02128* (2022).

[8] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, and et al. 2020. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.

[9] Saheed A Busari and Emmanuel Letier. 2017. Radar: A lightweight tool for requirements and architecture decision analysis. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 552–562.

[10] Victor R Basili1 Gianluigi Caldiera and H Dieter Rombach. 1994. The goal question metric approach. *Encyclopedia of software engineering* (1994), 528–532.

[11] Xiang Chen, Chaoyang Gao, Chunyang Chen, Guangbei Zhang, and Yong Liu. 2025. An Empirical Study on Challenges for LLM Application Developers. arXiv:2408.05002 [cs.SE] https://arxiv.org/abs/2408.05002

[12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. arXiv:2205.14135 [cs.LG] https://arxiv.org/abs/2205.14135

[13] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems* 35 (2022), 30318–30332.

[14] Georgi Gerganov. [n. d.]. GGML. https://github.com/ggerganov/ggml

[15] Georgi Gerganov. 2023. GGUF. https://github.com/ggerganov/ggml/blob/master/docs/gguf.md

[16] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2023. Knowledge distillation of large language models. *arXiv preprint arXiv:2306.08543* (2023).

[17] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. arXiv:2401.14196 [cs.SE] https://arxiv.org/abs/2401.14196

[18] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models. *arXiv preprint arXiv:2403.07714* (2024).

[19] Zixu Hao, Huiqiang Jiang, Shiqi Jiang, Ju Ren, and Ting Cao. 2024. Hybrid SLM and LLM for Edge-Cloud Collaborative Inference. *Proceedings of the Workshop on Edge and Mobile Foundation Models* (2024). https://api.semanticscholar.org/CorpusID:270405086

[20] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).

[21] Mojan Javaheripi, Sébastien Bubeck, Marah Abdin, Jyoti Aneja, Sebastien Bubeck, Caio César Teodoro Mendes, Weizhu Chen, Allie Del Giorno, Ronen Eldan, Sivakanth Gopi, et al. 2023. Phi-2: The surprising power of small language models. *Microsoft Research Blog* 1, 3 (2023), 3.

[22] Nan Jiang, Thibaud Lutellier, and Lin Tan. 2021. Cure: Code-aware neural machine translation for automatic program repair. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1161–1173.

[23] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.

[24] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 3045–3059. doi:10.18653/v1/2021.emnlp-main.243

[25] Xiang Lisa Li and Percy Liang. 2021. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, Online, 4582–4597. doi:10.18653/v1/2021.acl-long.353

[26] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.

[27] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Thirty-seventh Conference on Neural Information Processing Systems*. https://openreview.net/forum?id=1qvx610Cu7

[28] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1831–1847.

[29] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh Murthy, Liangwei Yang, Silvio Savarese, Juan Carlos Niebles, Huan Wang, Shelby Heinecke, and Caiming Xiong. 2024. APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets. arXiv:2406.18518 [cs.CL] https://arxiv.org/abs/2406.18518

[30] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, and et al. 2024. StarCoder 2 and The Stack v2: The Next Generation. arXiv:2402.19173 [cs.SE] https://arxiv.org/abs/2402.19173

[31] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664* (2021).

[32] NousResearch. 2023. Nous-Hermes-13b. https://huggingface.co/NousResearch/Nous-Hermes-13b

[33] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*. 311–318.

[34] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2023. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334* (2023).

[35] Nikhil Pinnaparaju, Reshinth Adithyan, Duy Phung, Jonathan Tow, James Baicoianu, and Nathan Cooper. [n. d.]. Stable Code 3B. https://huggingface.co/stabilityai/stable-code-3b

[36] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789* (2023).

[37] Yisheng Song, Ting Wang, Puyu Cai, Subrota K Mondal, and Jyoti Prakash Sahoo. 2023. A comprehensive survey of few-shot learning: Evolution, applications, challenges, and opportunities. *Comput. Surveys* 55, 13s (2023), 1–40.

[38] Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. 2023. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301* (2023).

[39] Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Haotian Hui, Weichuan Liu, Zhiyuan Liu, et al. 2024. Debugbench: Evaluating debugging capability of large language models. *arXiv preprint arXiv:2401.04621* (2024).

[40] Fali Wang, Zhiwei Zhang, Xianren Zhang, Zongyu Wu, Tzuhao Mo, Qiuhao Lu, Wanjing Wang, Rui Li, Junjie Xu, Xianfeng Tang, Qi He, Yao Ma, Ming Huang, and Suhang Wang. 2024. A Comprehensive Survey of Small Language Models in the Era of Large Language Models: Techniques, Enhancements, Applications, Collaboration with LLMs, and Trustworthiness. arXiv:2411.03350 [cs.CL] https://arxiv.org/abs/2411.03350

[41] Yiwei Wang, Muhao Chen, Nanyun Peng, and Kai-Wei Chang. 2024. DeepEdit: Knowledge Editing as Decoding with Constraints. arXiv:2401.10471 [cs.CL]

[42] Yaqing Wang, Quanming Yao, James T Kwok, and Lionel M Ni. 2020. Generalizing from a few examples: A survey on few-shot learning. *ACM computing surveys (csur)* 53, 3 (2020), 1–34.

[43] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, Anders Wesslén, et al. 2012. *Experimentation in software engineering*. Vol. 236. Springer.

[44] Fangzhou Wu, Ning Zhang, Somesh Jha, Patrick McDaniel, and Chaowei Xiao. 2024. A New Era in LLM Security: Exploring Security Concerns in Real-World LLM-based Systems. arXiv:2402.18649 [cs.CR] https://arxiv.org/abs/2402.18649

[45] Biwei Yan, Kun Li, Minghui Xu, Yueyan Dong, Yue Zhang, Zhaochun Ren, and Xiuzhen Cheng. 2024. On protecting the data privacy of large language models (llms): A survey. *arXiv preprint arXiv:2403.05156* (2024).

[46] Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. Berkeley Function Calling Leaderboard. https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html.

[47] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).

[48] Kechi Zhang, Huangzhao Zhang, Ge Li, Jia Li, Zhuo Li, and Zhi Jin. 2023. Toolcoder: Teach code generation models to use api search tools. *arXiv preprint arXiv:2305.04032* (2023).

[49] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).

[50] Lucen Zhong, Zhengxiao Du, Xiaohan Zhang, Haiyi Hu, and Jie Tang. 2025. ComplexFuncBench: Exploring Multi-Step and Constrained Function Calling under Long-Context Scenario. *arXiv preprint arXiv:2501.10132* (2025).

[51] Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. 2024. A survey on model compression for large language models. *Transactions of the Association for Computational Linguistics* 12 (2024), 1556–1577.